Volume 02 | The Prediction Request, JSON Contract, and Line Allocation

Walk through API input validation, inference orchestration and output.

1. Browser request shape

The dashboard JS posts JSON to `/api/predict`: `{"station":"Rajiv Chowk","date":"YYYY-MM-DD","time":"09:00","line":""}`. The endpoint also supports GET query parameters. For a POST whose Content-Type contains `application/json`, invalid JSON yields a 400 response and a parsed non-dict JSON value is also rejected. Other request forms take query params.

2. Handler versus worker helper

`api_predict` is async. It awaits request parsing and delegates `_predict_data(data)` to `run_in_threadpool`. The helper does the orchestration synchronously. This protects the event loop from synchronous Pandas/sklearn work; it does not accelerate the model. Threadpool capacity and CPU/memory still bound throughput.

3. Input normalization and time

`_predict_data` stringifies and strips the station, removes footnote marks, and gets date/time defaults. `_parse_datetime` parses ISO date and `HH:MM`; parse failure returns a 400 JSON message. It constructs a naive datetime from local date/hour/minute; the application assumes Delhi local time, but the datetime object itself has no `tzinfo`. Date context and `is_service_operating` are then derived.

Closed service is explicit: if outside 05:00 inclusive to 23:30 exclusive, prediction is zero and scheduling is replaced by a `SERVICE_CLOSED` headline, no trains/headway, and a closure explanation. This is a modeled window, not proof of current official timetable.

4. Prediction and schedule stages

Station metadata -> `predict_footfall_with_minute_interpolation` and `predict_hourly_footfall` -> 19-point `predict_station_full_day_profile` -> `optimize_station_schedule`. The scheduling function uses station-level line split; a selected line narrows headline recommendation. If interchange and no line filter, route aggregates demand/capacity across every modeled line so total demand is not compared to only one line's trains.

For station summary, API sums recommended and existing clearance capacity, totals trains/hour/fleet and computes demand-weighted average wait. Note this aggregation is still a scenario calculation based on assumptions. Capacity utilization is `estimated demand / recommended boarding-clearance capacity * 100`; it is not onboard occupancy.

5. Travel-time stage

`shortest_times_from(travel_graph, station)` runs from every source line-state. The route then attaches a corridor label using the station table's primary `Line`; this display label is not the actual line sequence of the shortest path. A future API improvement would return path legs/line changes from the graph search rather than a single station corridor label.

6. JSON response map

- `status`, `station`, `station_info`.
- `datetime`: date/day/time/hour/minute, peak flag, service status.
- `calendar_context`: occasion, workday/public-holiday state, selected demand band, daily peak.
- `footfall`: interpolated instant, hourly estimate, daily total, peak time/value.
- `scheduling`: headline recommendation, per-line recommendations, interchange state.
- `profile_24h`: hourly labels/predictions/baseline/line curves.

- `travel_times`: destination, estimated minutes, one display corridor label.
- `data_provenance`: synthetic forecast, project topology, configurable operations, estimated line split.

7. Contract weaknesses

No Pydantic model currently describes body types or OpenAPI request schema. Unknown stations can flow into fallbacks and index 0 in the model metadata. Unknown line filters silently leave the default recommendation. Date range is not limited by API contract; future calendar can label dates with sparse event knowledge. These are concrete validation improvements.

Exercises

Trace both GET and POST paths. Identify exactly when status 400 is returned. Propose a typed `PredictionRequest` with station/date/time/line and explain why schema validation is different from business validation. Explain why a closed-time API should not label the value as observed zero ridership.